

ITUNIVERSITY — MODULE M1 · SÉANCE 2

L'asynchrone : les coroutines Kotlin

La concurrence sans douleur — suspend, launch, async/await
Mini-TP « Prédire puis vérifier »



Pourquoi l'asynchrone est inévitable en mobile

1 Un seul thread pour l'interface

Il dessine l'écran et reçoit les gestes. S'il attend, l'écran gèle — et le système propose de tuer l'application (ANR).

2 Les attentes sont partout

Réseau, base de données, fichiers, calculs. En mobile, on attend en permanence — il faut organiser l'attente, pas l'éviter.

3 La réponse : les coroutines

Des tâches légères qui se suspendent pendant l'attente et rendent le thread — puis reprennent exactement où elles en étaient.

La règle d'or du mobile : rien ne doit jamais bloquer le thread de l'interface.



Qu'est-ce qu'une coroutine ? Le client, pas le guichet

LE THREAD : un guichet

Le client qui attend son fournisseur reste vissé au guichet.
Guichet bloqué → la file s'allonge → l'écran gèle.

≈ 1 Mo de pile chacun · au-delà de quelques dizaines actifs, la commutation coûte plus qu'elle ne rapporte

LA COROUTINE : un client

Le client qui doit attendre s'écarte avec un ticket ; le guichet sert les autres ; il revient quand on l'appelle.

quasi gratuite · 100 000 possibles, multiplexées sur un petit pool de threads (≈ le nombre de cœurs)

	Thread	Coroutine
Fourni par	Le système d'exploitation	Kotlin — par-dessus les threads
Coût	≈ 1 Mo chacun ; parallélisme réel plafonné aux cœurs (≈ 8), commutation système coûteuse	Quasi gratuite ; bascule = simple opération mémoire, des milliers sur 8 threads
Pendant une attente	Dort en tenant sa place	Descend du thread et le libère — puis reprend où elle s'était arrêtée

La suite de la séance en une phrase : suspend déclare la capacité de pause · la coroutine est le véhicule qui la porte · launch et async mettent le véhicule en route.



suspend : la fonction qui sait attendre

Une fonction qui attend — sans bloquer

```
suspend fun chargerProduits(): List<String> {  
    println("Téléchargement...")  
    delay(1000)    // suspend, ne bloque pas  
    return listOf("Vanille", "Café")  
}  
  
// Se lit de haut en bas,  
// comme du code ordinaire.  
// Pas de callback, pas d'imbrication.
```

S'appelle depuis une coroutine

```
fun main() = runBlocking {  
    val produits = chargerProduits()  
    println(produits)  
}  
  
// runBlocking : crée la coroutine  
// de nos programmes console.  
// Dans Android : le ViewModel  
// s'en chargera (séance 6).
```

delay suspend la coroutine — Thread.sleep bloquerait le thread. Toute la différence est là.



suspend · launch · async — trois mots, deux catégories

suspend

Nature

Mot-clé du langage — une CAPACITÉ : la fonction peut se mettre en pause sans bloquer.

Retourne

La valeur de la fonction, normalement.

Quand

Toute fonction qui attend : réseau, base de données, delay.

```
suspend fun charger(): List<P>
```

launch

Nature

Fonction de `kotlinx.coroutines` — un DÉMARREUR : « lancer et oublier ».

Retourne

Un `Job` (annuler, attendre la fin) — pas de valeur.

Quand

Effet sans résultat attendu : journaliser, synchroniser en fond.

```
launch { synchroniser() }
```

async

Nature

Fonction de `kotlinx.coroutines` — un DÉMARREUR : « lancer et récupérer ».

Retourne

Un `Deferred<T>` — une promesse, encaissée avec `.await()`.

Quand

Résultat attendu, souvent en parallèle d'autres tâches.

```
val d = async { compter() }
```

Une capacité (suspend) — deux démarreurs (launch, async). Ne pas les ranger dans la même boîte.

À la banque : suspend = savoir patienter au guichet · launch = « va ranger le bureau » (aucun rapport attendu) · async = « va vérifier mon compte » (je viendrai chercher le chiffre avec `await`).



Qui appelle une fonction suspend ? Quatre façons — un seul critère

1 Appel direct — déjà dans une coroutine

```
fun main() = runBlocking {  
    val liste = chargerProduits() // c'est tout  
}
```

Résultat tout de suite, dans l'ordre. Ni launch ni async nécessaires.

2 Depuis un launch — en parallèle, sans résultat

```
launch {  
    chargerProduits() // résultat ignoré  
}
```

Effet en arrière-plan : journaliser, rafraîchir, synchroniser.

3 Depuis un async — en parallèle, avec résultat

```
val d = async { chargerProduits() }  
val liste = d.await()
```

Le résultat, mais pendant que d'autres tâches courent aussi.

4 Depuis une autre fonction suspend

```
suspend fun synchroniserTout() {  
    val p = chargerProduits() // légal  
}
```

La chaîne remonte : tout en haut, une coroutine porte le tout.

Le critère n'est pas dans la fonction — il est dans votre besoin :
résultat immédiat → direct . pas de résultat → launch · résultat en parallèle → async.

Et runBlocking ? Un démarreur de plus — mais qui BLOQUE le thread : réservé au pont console/tests. Jamais dans Android (les scopes s'en chargeront, séance 6).



Lancer plusieurs coroutines : launch et async

launch — lancer et oublier

```
fun main() = runBlocking {  
    println("A")  
    launch {  
        delay(500)  
        println("B")  
    }  
    println("C")  
}  
  
// Quel ordre d'affichage ?  
// C'est le programme 1 du mini-TP.
```

async / await — lancer et récupérer

```
fun main() = runBlocking {  
    val a = async { delay(1000); "produits" }  
    val b = async { delay(1500); "collectes" }  
    println(a.await() + " + " + b.await())  
}  
  
// Les deux courent EN MÊME TEMPS :  
// durée totale = max(1000, 1500)  
//                ≠ 1000 + 1500
```

Le piège de la semaine : l'endroit où l'on place await change tout — programme 3 du mini-TP.



Démonstration chronométrée : séquentiel vs concurrent

Séquentiel — ≈ ?

```
val t = measureTimeMillis {  
    val a = async { delay(1000); "P" }  
                .await()    // attend ici  
    val b = async { delay(1500); "C" }  
                .await()    // puis là  
    println("$a $b")  
}  
println("Durée : $t ms")
```

Concurrent — ≈ ? s

```
val t = measureTimeMillis {  
    val a = async { delay(1000); "P" }  
    val b = async { delay(1500); "C" }  
    // les deux courent déjà...  
    println(a.await() + " " + b.await())  
}  
println("Durée : $t ms") /
```

Deux codes presque identiques — c'est la position des await qui fait la concurrence.



Pour la suite du module : Dispatchers et Flow, en survol

Dispatchers — où s'exécute la coroutine

- Dispatchers.Main : le thread de l'interface
- Dispatchers.IO : réseau, fichiers, base de données
- À retenir : « le travail lourd ailleurs, le résultat sur Main »

Flow — la donnée qui arrive en continu

- Pas une valeur : un flux de valeurs dans le temps
- Se collecte ; chaque émission redéclenche le traitement
- Alimentera l'interface réactive (StateFlow, séance 6)

Aujourd'hui : savoir que cela existe. Le reste viendra en contexte, dans Android.



Mini-TP 2 · « Prédire puis vérifier »

1 Prédire

Test « S2 · Prédications » : ordre exact des affichages + durée estimée, pour chacun des 3 programmes.

2 Exécuter et expliquer

Comparer aux prédictions ; expliquer chaque écart par écrit.

3 Transformer

Programme 2 : le rendre séquentiel et mesurer. Programme 3 : le rendre concurrent et mesurer. Un déplacement d'await chacun.

4 Voie ouverte — juger l'IA

Faire expliquer UN écart par l'IA, puis reformuler avec ses propres mots (3 lignes au journal).

